

PROJEKT ZALICZENIOWY TAS5

Opis testów automatycznych Selenium WebDriver

Dokument przygotowany na podstawie aktualnych testów w projekcie Pepco. Zawiera cel, kroki oraz zakres weryfikacji dla każdej klasy testowej.

Projekt: TAS5_praca_po_poprawkach

Technologie: Java, Maven, JUnit 5, Selenium WebDriver, Page Object Model, DataFaker

Data wygenerowania: 2026-05-27

Opis testów

OPIS TESTOW W PROJEKCIE TAS5

Projekt zawiera testy automatyczne Selenium WebDriver uruchamiane przez Maven i JUnit 5. Testy korzystają z Page Object Model, PageFactory, jawnych oczekiwan WebDriverWait oraz danych testowych z plików CSV, MethodSource i DataFaker.

1. CollectionTest

Komenda:

```
./mvnw -Dtest=CollectionTest test
```

Cel testu:

Sprawdzenie przejścia z kolekcji produktów do produktu, a następnie do informacji o zwrotach.

Test: thirdProductOpensReturnPolicy

Kroki:

1. Test pobiera dane z CollectionData#returnPolicyCollections.
2. Otwiera wskazaną kolekcję, np. /collections/dom/oswietlenie albo /collections/dziecko/chlopiec-92-134.
3. Akceptuje cookies.
4. Klika trzeci produkt na stronie kolekcji.
5. Sprawdza, czy URL zawiera /products/.
6. Na stronie produktu klika informację "30 dni na zwrot".
7. Otwiera regulamin lub stronę zasad zwrotu.
8. Sprawdza, czy adres URL zawiera oczekiwany fragment.
9. Czeką aż strona się ustabilizuje.
10. Wykonuje screenshot dla danego wariantu testowego.

Co jest sprawdzane:

- czy kolekcja zawiera co najmniej trzy produkty,
- czy kliknięcie produktu otwiera stronę produktu,
- czy z produktu da się przejść do informacji o zwrotach,
- czy otwierana jest poprawna strona/regulamin zwrotów.

2. DmpEventsTest

Komenda:

```
./mvnw -Dtest=DmpEventsTest test
```

Cel testu:

Sprawdzenie, czy po zaakceptowaniu cookies odpalają się requesty Mediarithmics oraz czy zawierają site_token=pepco-24-pl.

Test: mediarithmicsDiagnostic

Kroki:

1. Test pobiera dane z DmpData#pages.
2. Uruchamia się bez trybu incognito, ponieważ tracking może wymagać zapisanych cookies.
3. Otwiera testowaną stronę, np. stronę główną albo stronę kolekcji.
4. Akceptuje wszystkie cookies.
5. Odświeża stronę po zapisaniu zgody.
6. Czeką na requesty DMP/Mediarithmics.
7. Pobiera requesty z logów performance Chrome.
8. Sprawdza, czy pojawił się pixel Mediarithmics.

9. Rozróżnia visits/pixel oraz check_cookie/pixel.
10. Wypisuje w konsoli:
 - liczbę requestów Mediarithmics,
 - status pixela,
 - czy odpalił się visits/pixel,
 - czy odpalił się check_cookie/pixel,
 - statusy HTTP, jeśli Selenium jest udostępniony,
 - site_token.
11. Sprawdza, czy lista requestów nie jest pusta.
12. Sprawdza, czy pixel się odpalił.
13. Sprawdza, czy występuje site_token=pepco-24-pl.
14. Jeśli status HTTP jest dostępny, sprawdza czy zawiera 200.
15. Wykonuje screenshot diagnostyczny.

Uwaga:

Chrome/Selenium nie zawsze udostępnia status HTTP dla beaconów/pixeli. Dlatego test traktuje status HTTP jako informację diagnostyczną, jeśli jest dostępna.

3. HomeCarouselPinnedProductTest

Komenda:

```
./mvnw -Dtest=HomeCarouselPinnedProductTest test
```

Cel testu:

Sprawdzenie ścieżki ze strony głównej: karuzela, CTA, pin produktu na zdjęciu i przejście na stronę produktu.

Test: carouselCtaPinnedProductOpensPdp

Kroki:

1. Otwiera stronę główną pepco.pl.
2. Czeką na załadowanie strony.
3. Akceptuje cookies.
4. Przechodzi po karuzeli na stronie głównej.
5. Klika CTA z sekcji na stronie głównej.
6. Klika pin produktu widoczny na zdjęciu.
7. Klika produkt pokazany w panelu pinu.
8. Czeką na przejście do strony produktu.
9. Sprawdza, czy URL zawiera /products/.
10. Wykonuje screenshot home_carousel_pinned_product.

Co jest sprawdzane:

- czy karuzela jest interaktywna,
- czy CTA na stronie głównej działa,
- czy pin produktu na zdjęciu otwiera panel produktu,
- czy kliknięcie produktu z pinu prowadzi do strony PDP.

4. NewsletterFakerTest

Komenda:

```
./mvnw -Dtest=NewsletterFakerTest test
```

Cel testu:

Sprawdzenie zapisu do newslettera z użyciem adresu e-mail wygenerowanego przez DataFaker.

Test: subscribesToNewsletterWithFakerEmail

Kroki:

1. Otwiera stronę główną pepco.pl.
2. Czeką na załadowanie strony.
3. Akceptuje cookies.
4. Generuje adres e-mail przez DataFaker.
5. Wypisuje wygenerowany e-mail w konsoli.
6. Wpisuje e-mail do formularza newslettera.
7. Zaznacza zgodę marketingową.
8. Klika przycisk "Zapisz się".
9. Wykonuje screenshot po wysłaniu formularza.
10. Czeką na komunikat potwierdzający zapis.
11. Wykonuje drugi screenshot z komunikatem potwierdzenia.

Co jest sprawdzane:

- czy można wpisać wygenerowany e-mail,
- czy checkbox zgody został zaznaczony,
- czy formularz newslettera został wysłany,
- czy pojawia się komunikat potwierdzający zapis.

Uwaga:

Formularz newslettera może być czasem blokowany przez mechanizmy antyspamowe albo backend newslettera. Wtedy test może pokazać, że formularz został wypełniony, ale komunikat potwierdzenia nie pojawił się.

5. PepcoClubTest**Komenda:**

```
./mvnw -Dtest=PepcoClubTest test
```

Cel testu:

Sprawdzenie linków do aplikacji Pepco w App Store i Google Play.

Test: appStorePageOpensAfterClick

Kroki:

1. Test pobiera dane z PepcoClubData#appStoreLinks.
2. Otwiera stronę /pepco-club.
3. Akceptuje cookies.
4. Sprawdza, czy URL zawiera pepco-club.
5. Sprawdza, czy tytuł strony Pepco Club jest widoczny.

6. Klika link App Store albo Google Play.
7. Przelacza sie na najnowsza karte przegladarki.
8. Czeki na zaladowanie strony sklepu.
9. Sprawdza, czy URL zawiera oczekiwany adres:
 - apps.apple.com/pl/app/pepco-deals-and-vouchers/id6757158188
 - play.google.com/store/apps/details?id=eu.pepco.app
10. Wykonuje screenshot strony sklepu.

Co jest sprawdzane:

- czy strona Pepco Club sie otwiera,
- czy linki do sklepow aplikacji sa klikalne,
- czy prowadzi do poprawnych zewnetrznych stron,
- czy dziala obsluga nowej karty.

6. RandomCityFromFakerTest

Komenda:

```
./mvnw -Dtest=RandomCityFromFakerTest test
```

Cel testu:

Sprawdzenie wyszukiwarki sklepow z losowym miastem wybranym przez DataFaker.

Test: randomCityFromFaker

Kroki:

1. Otwiera strone /store-locator.
2. Akceptuje cookies.
3. Losuje miasto z listy: Warszawa, Krakow, Gdansk, Wroclaw, Poznan, Lodz.
4. Wypisuje testowane miasto w konsoli.
5. Wpisuje miasto w wyszukiwarke sklepow.
6. Sprawdza, czy pole wyszukiwania nie jest puste.
7. Sprawdza, czy pole zawiera szukane miasto.
8. Sprawdza, czy pojawil sie wynik wyszukiwania dla miasta.
9. Otwiera pierwszy sklep z wynikow dla tego miasta.
10. Ustawia sklep jako "Twój sklep".
11. Sprawdza, czy URL po kliknieciu nie jest pusty.
12. Wykonuje screenshot random_city_favorite_store.

Co jest sprawdzane:

- dzialanie wyszukiwarki sklepow,
- obsluga danych generowanych przez Faker,
- otwieranie szczegolow sklepu,
- ustawienie sklepu jako ulubionego.

7. StoreLocatorCsvTest

Komenda:

```
./mvnw -Dtest=StoreLocatorCsvTest test
```

Cel testu:

Sprawdzenie wyszukiwarki sklepów dla kilku miast z pliku CSV.

Test: searchStoreByCityFromCsv

Kroki:

1. Test pobiera dane z pliku src/test/resources/test_data/store_locator_data.csv.
2. Dla każdego miasta otwiera stronę /store-locator.
3. Akceptuje cookies.
4. Wpisuje miasto w wyszukiwarke sklepów.
5. Sprawdza, czy pole wyszukiwania nie jest puste.
6. Sprawdza, czy pole zawiera oczekiwany tekst miasta.
7. Sprawdza, czy pojawiły się wyniki wyszukiwania dla miasta.
8. Otwiera pierwszy sklep z wyników dla danego miasta.
9. Ustawia sklep jako "Twój sklep".
10. Wykonuje screenshot store_locator_csv_favorite_nazwa.
11. Wypisuje testowane miasto w konsoli.

Co jest sprawdzane:

- data driven testing z plikiem CSV,
- wyszukiwanie sklepów dla wielu danych,
- otwieranie szczegółów sklepu,
- ustawienie sklepu jako ulubionego.

8. TrendyTest

Komenda:

```
./mvnw -Dtest=TrendyTest test
```

Cel testu:

Sprawdzenie strony "Oferta gazetkowa" i produktów widocznych w tej sekcji.

Test: trendyPageOpens

Kroki:

1. Otwiera /trendy/oferta-gazetkowa.
2. Akceptuje cookies.
3. Sprawdza, czy URL zawiera oferta-gazetkowa.
4. Wykonuje screenshot trendy_page.

Test: productsAreVisible

Kroki:

1. Otwiera /trendy/oferta-gazetkowa.
2. Akceptuje cookies.
3. Sprawdza, czy na stronie są linki produktów zawierające /products/.
4. Wykonuje screenshot trendy_products.

Test: clickProductOpensPdp

Kroki:

1. Otwiera /trendy/oferta-gazetkowa.
2. Akceptuje cookies.
3. Klika pierwszy produkt z listy.
4. Sprawdza, czy kliknięcie się udało.
5. Sprawdza, czy URL zawiera /products/.

Co jest sprawdzane:

- wejście na stronę oferty gazetkowej,
- widoczność produktów,
- przejście z listy produktów na stronę produktu.

9. BaseTest

BaseTest nie jest samodzielnym testem biznesowym, ale jest klasa bazowa dla pozostałych testów.

Zawiera:

1. Konfigurację ChromeDriver.
2. Uruchamianie Chrome w trybie incognito dla większości testów.
3. Możliwość wyłączenia incognito przez nadpisanie metody useIncognito().
4. Konfigurację logów performance Chrome.
5. Zamykanie przeglądarki po każdym teście.
6. Pomocniczą metodę takeScreenshot().

PODSUMOWANIE

Projekt zawiera testy:

- CollectionTest
- DmpEventsTest
- HomeCarouselPinnedProductTest
- NewsletterFakerTest
- PepcoClubTest
- RandomCityFromFakerTest
- StoreLocatorCsvTest
- TrendyTest

Zastosowane techniki:

- Selenium WebDriver,
- JUnit 5,
- Page Object Model,
- PageFactory,
- WebDriverWait,
- data driven testing przez MethodSource,
- data driven testing przez CSV,
- DataFaker,

- screenshoty po ważnych akcjach,
- diagnostyka requestów Mediarithmics przez logi performance Chrome.

Uwaga: test newslettera może zależeć od zewnętrznego backendu, mechanizmów antyspamowych lub walidacji formularza po stronie serwisu.